

MILAN — Editing Spurious Features

Figures, Results & Visualizations

Reproduction of Hernandez et al., ICLR 2022 · 10% overlay setting

This document collects every figure from the project — the method, the dataset, the paper's own results, our reproduction, and the three additional experiments — and explains, for each one, what it shows, how to read it, and why it matters. All quantitative results use the 10% training-overlay setting, the configuration that gives the clearest editing signal.

Contents

METHOD

- The MILAN editing pipeline

DATASET

- Spurious-text Imagenette (how the shortcut is built)

REFERENCE — THE PAPER

- Paper Fig 7: MILAN names the text neuron
- Paper Fig 8: ablation recovers robustness

OUR REPRODUCTION (10% OVERLAY)

- Results at a glance
- Our Fig 7: the text neurons we found
- Our Fig 8: the editing curve

ADDITIONAL EXPERIMENT 1

- Does editing generalize? (VGG16)
- Text-neuron fraction across architectures

ADDITIONAL EXPERIMENT 2

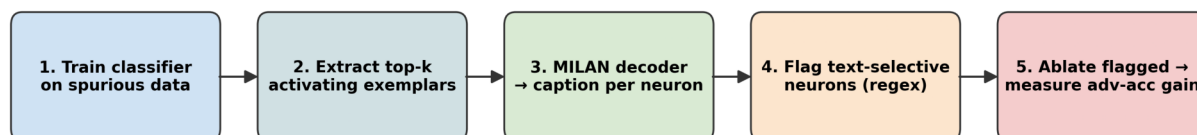
- Where does the shortcut live? (layer depth)

ADDITIONAL EXPERIMENT 3

- Is CLIP more robust? (zero-shot ViT-B/32)

The MILAN editing pipeline

MILAN pipeline for editing spurious features



(steps 1–5 form one self-contained loop; we extend step 1 with VGG16 / CLIP, and add per-layer analysis at step 4)

MILAN describes a neuron by feeding its top-activating image patches (its "exemplars") into a pretrained, image-conditioned language model trained on 60k human neuron descriptions (the MILANNOTATIONS dataset). The output is a free-form English caption per neuron — e.g. "words and letters" or "dog faces" — rather than a label drawn from a fixed concept list, which is exactly what sets MILAN apart from earlier interpretability methods.

Section 7 of the paper turns those captions into the editing loop shown above: train a classifier on spurious data, caption every neuron, flag the text-selective ones with a simple keyword filter, zero (ablate) them, and re-measure adversarial accuracy.

No retraining or gradient surgery is involved. Because the captions are human-readable, the description itself tells you which neurons to remove — that is what makes an otherwise opaque network editable "by name", and it is the capability the rest of this document tests.

Spurious-text Imagenette (how the shortcut is built)

Spurious-Text Imagenette: how the corner-text shortcut is created

Clean Imagenette
(no overlay)



Training set
— class-name overlay



Adversarial test
(wrong-class overlay)

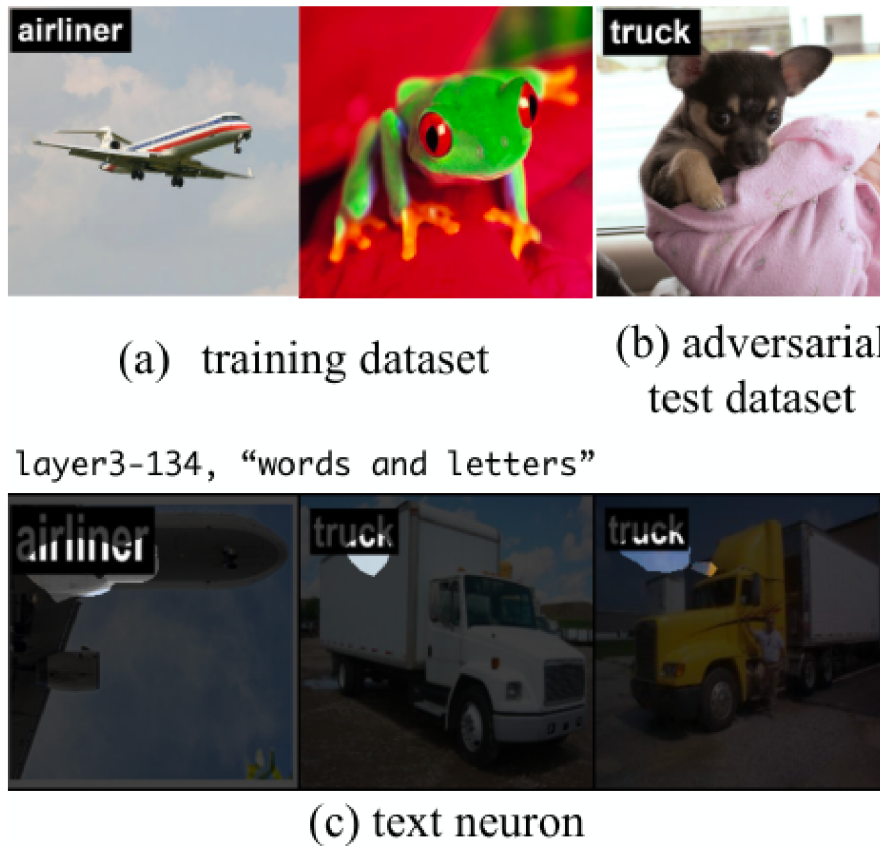


Top row: clean Imagenette images (10 ImageNet classes), with no overlay.

Middle row: the training set. A fraction (10%) of images have their CORRECT class name painted in the top-left corner. Because that text is perfectly predictive of the label, a model is tempted to read it instead of looking at the object — a spurious shortcut planted on purpose.

Bottom row: the adversarial test set, where every image carries a WRONG class name. This is the crux of the design: a model that truly recognizes objects is unaffected, but a model that learned to read the corner text is now actively misled, so its accuracy collapses. The size of that collapse is precisely the robustness gap the editing experiment sets out to repair.

Paper Fig 7: MILAN names the text neuron

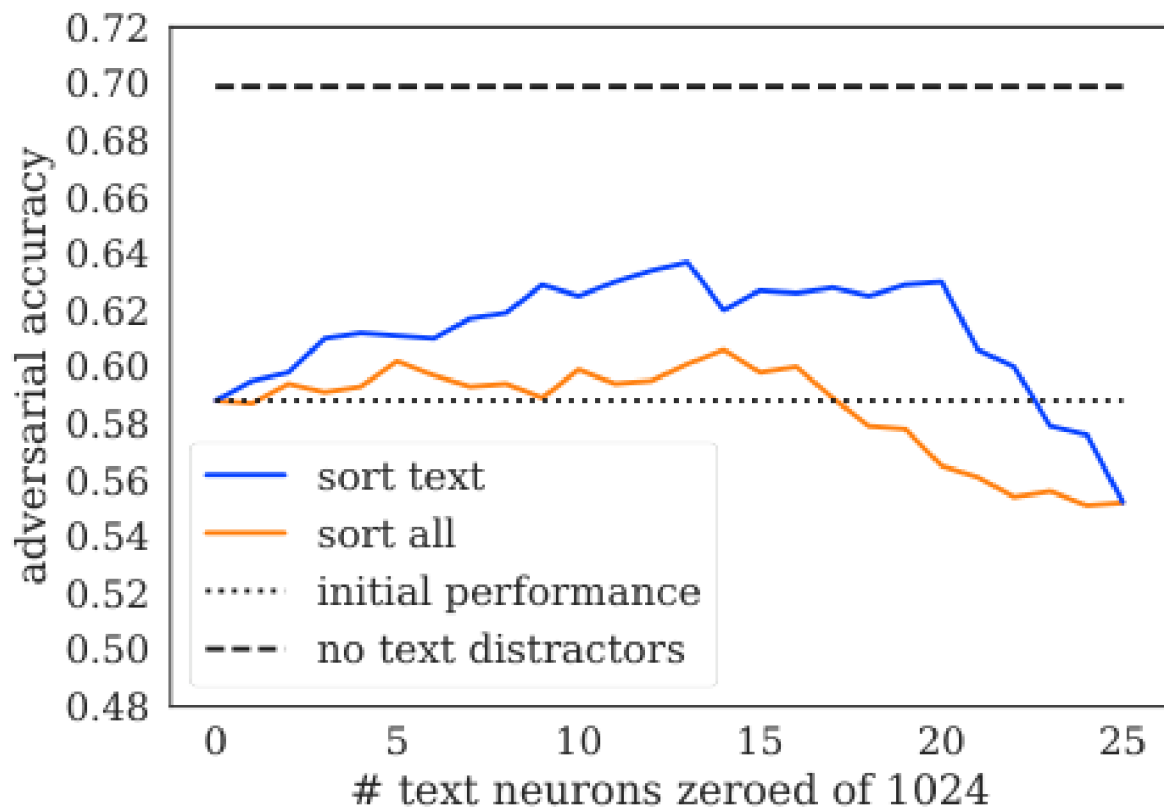


From Hernandez et al. (2022). Panels (a) and (b) show training examples with the corner text and the adversarial test set with wrong-class text.

Panel (c) shows the top exemplars of a single neuron (layer3-134) that MILAN captions "words and letters" — the spurious feature, named in plain English.

This is the qualitative claim we set out to reproduce: rather than staying hidden, the shortcut is surfaced by MILAN as one human-readable phrase. That phrase is what makes the downstream editing possible — you can search captions for "text" and immediately have a list of neurons to remove.

Paper Fig 8: ablation recovers robustness



The paper's headline editing result. The x-axis is the number of neurons zeroed; the y-axis is adversarial accuracy.

The blue "sort text" curve ablates the neurons MILAN captioned as text and climbs from ~58% to ~63%, clearly beating the orange "sort all" baseline that removes neurons by overall importance. The dashed line marks the no-text-distractor ceiling.

The important point is not just that accuracy rises, but that the caption-guided ordering beats an importance-based one: deleting the MILAN-named neurons restores most of the lost robustness with no retraining. This is the curve we reproduce on our own data over the next three pages.

Results at a glance

Summary of reproduction + extensions

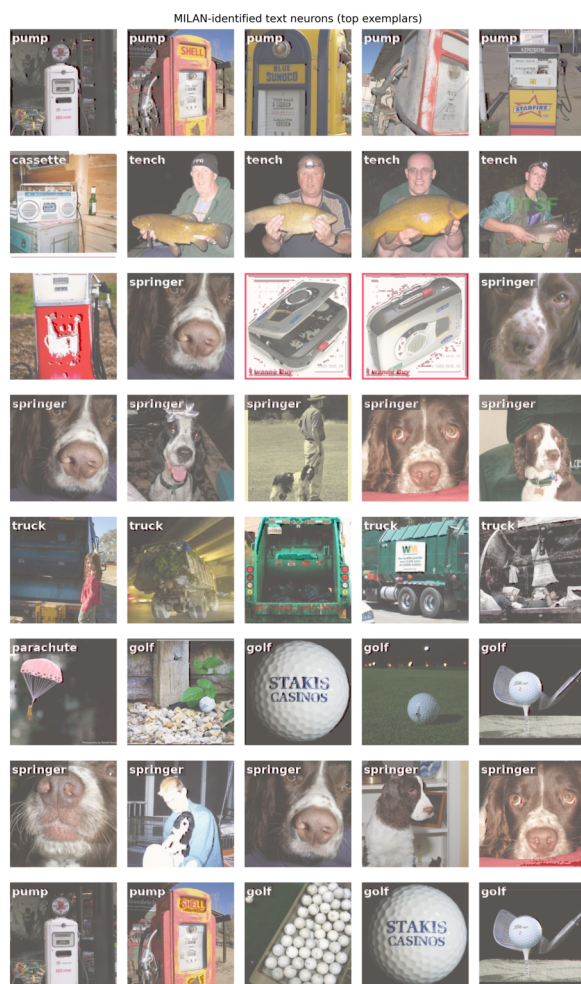
Experiment	Clean val	Adv test	Text neurons	Key finding
Baseline reproduction (ResNet18)	75.4%	51.5%	216 / 1024 (21.1%)	Editing recovers robustness
Exp 1 — VGG16 generalization	69.0%	19.8%	357 / 1472 (24.3%)	More reliant on shortcut
Exp 2 — ResNet18 layer-depth	—	—	9→27→37→77→66	Peaks at layer1 (3.0×), then declines
Exp 3 — CLIP ViT-B/32 (zero-shot)	—	—	61 / 768 (7.9% max)	≈2.7× more robust

A summary of the reproduction plus the three extensions, all on the 10% overlay setting.

Baseline ResNet18: 75.4% clean accuracy, 51.5% adversarial accuracy, and MILAN flags 216/1024 (21.1%) of neurons as text-selective. Ablating those neurons recovers adversarial accuracy and beats both the random and importance baselines, so the paper's core editing claim reproduces.

The three extension rows probe how far that result generalizes — to a second architecture (VGG16), across network depth, and to a vision-language model (CLIP). Each is detailed on the following pages, and they all point in a consistent direction.

Our Fig 7: the text neurons we found

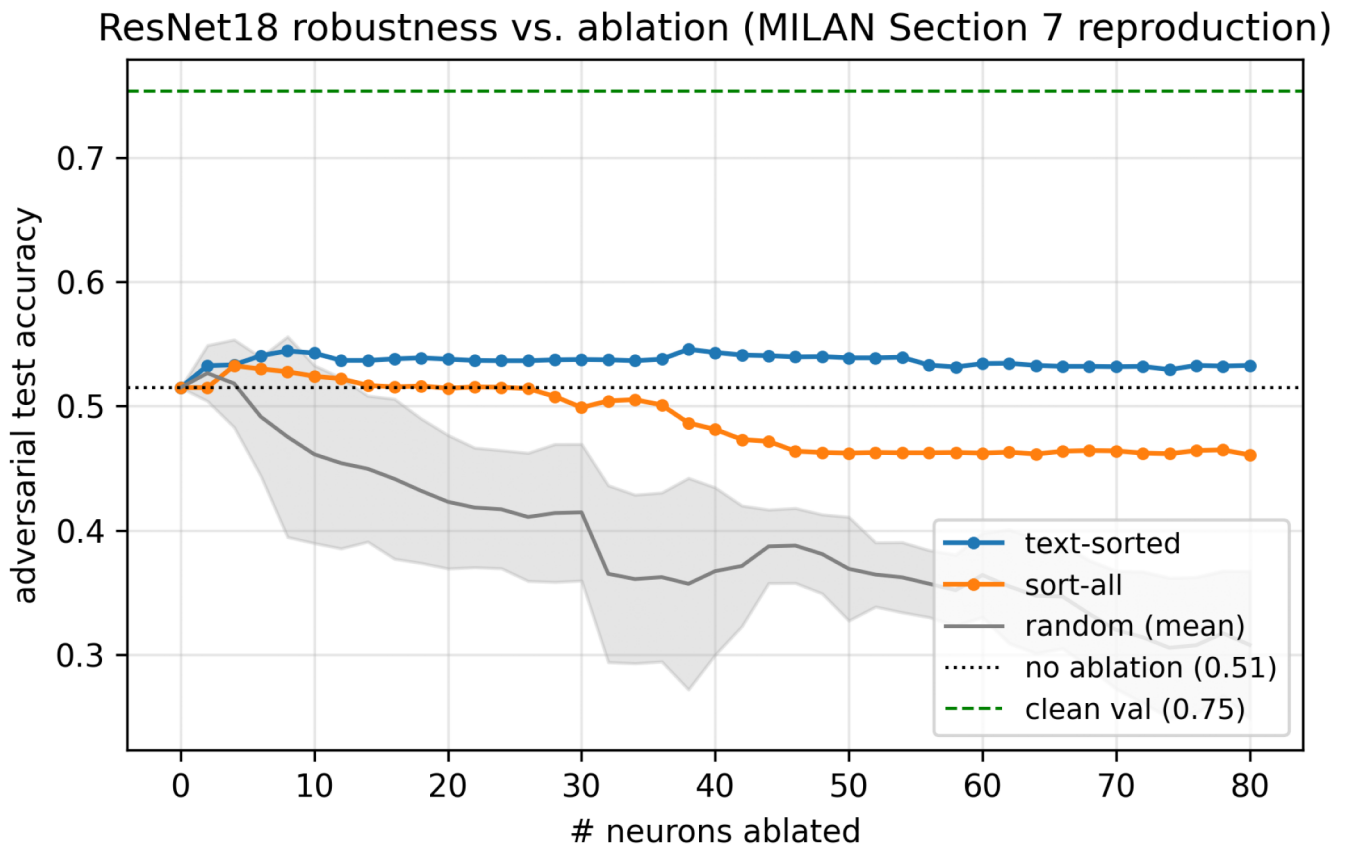


Eight MILAN-flagged text neurons from our ResNet18. Each row is one neuron, and the row label on the left is MILAN's caption for it.

The painted class words visible inside the exemplar thumbnails ("pump", "truck", "springer", ...) are exactly the corner text these neurons respond to.

That correspondence is the qualitative sanity check behind the whole method: the flagged units are genuinely reading text, not arbitrary neurons that happened to match the keyword filter. Because the captions are accurate here, they can be trusted as editing targets — which the next page puts to the test quantitatively.

Our Fig 8: the editing curve

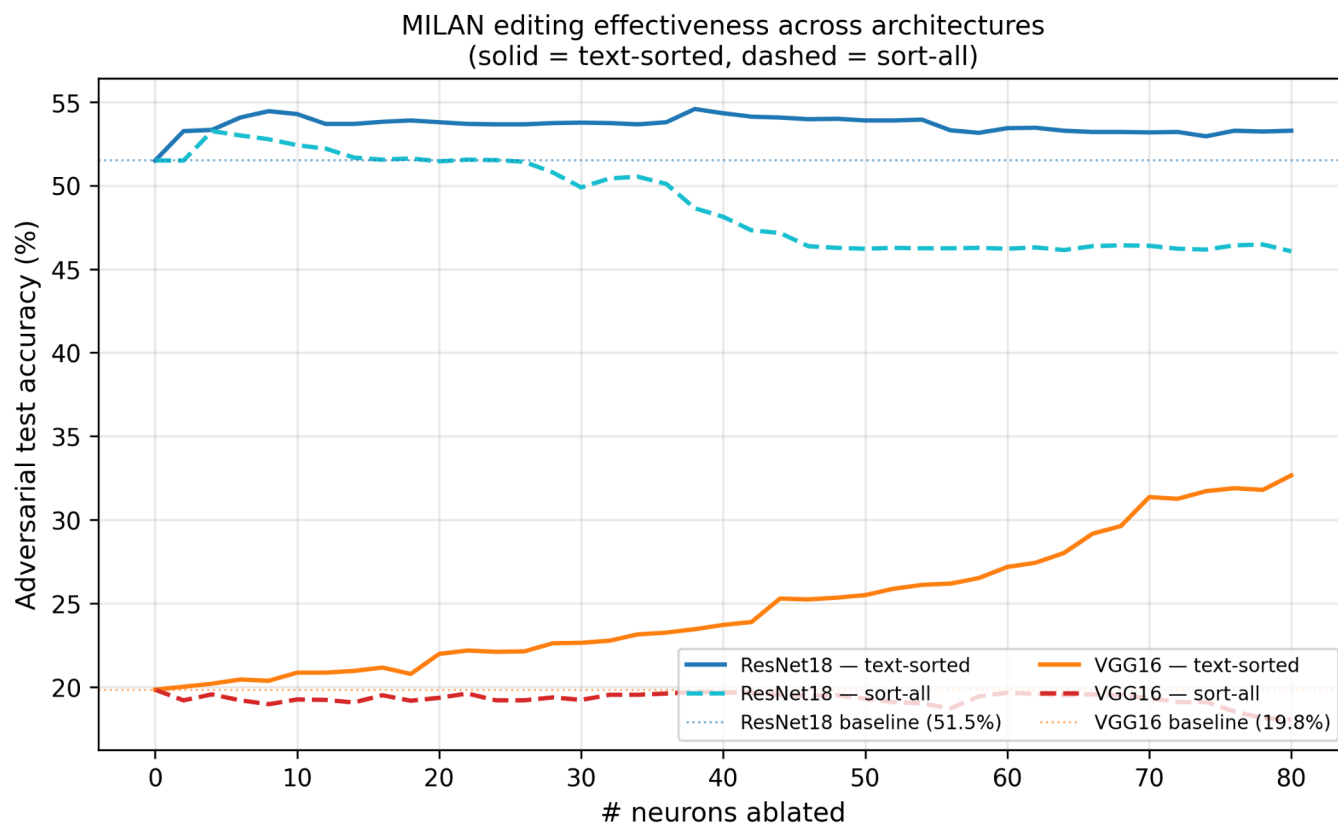


Our reproduction of the editing result. From the 51.5% no-ablation baseline (dotted), zeroing MILAN's text neurons (blue, "text-sorted") recovers adversarial accuracy to ~55% and holds it.

Crucially, ablating the SAME number of neurons by importance (orange, "sort-all") or at random (grey, with the shaded ± 1 std band) instead DEGRADES accuracy, to ~46% and ~37%. The green dashed line is clean accuracy (0.75).

This is the key control in the whole project: it is the MILAN captions, not the mere act of removing neurons, that recover robustness — without them, the same number of deletions makes the model worse. Text-sorted is the only curve that stays above its starting line, so the captions are doing real work.

Does editing generalize? (VGG16)

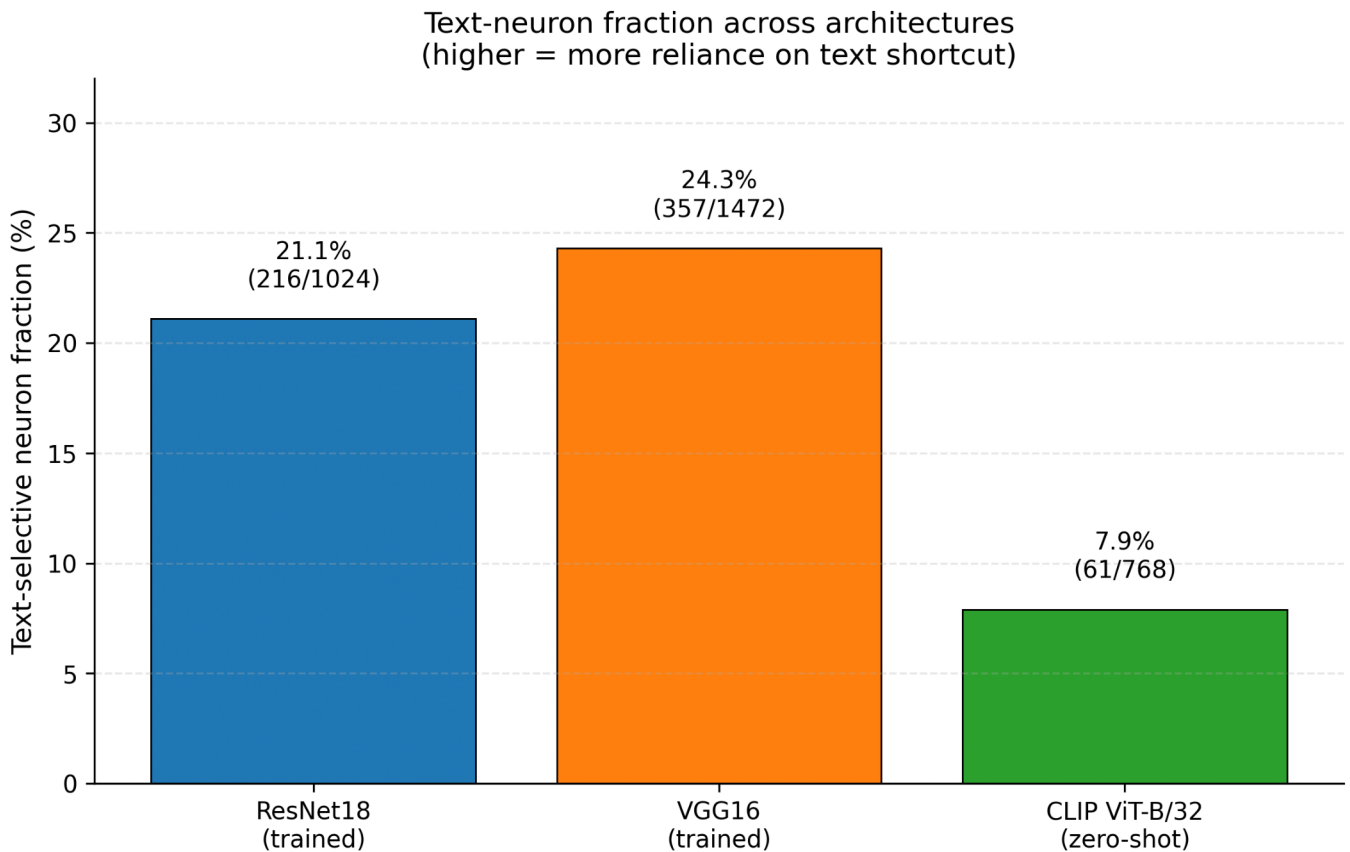


Per-architecture ablation curves, with solid = text-sorted (MILAN-guided) and dashed = sort-all (importance-ordered).

VGG16 is MORE shortcut-reliant than ResNet18 — its no-ablation adversarial accuracy is only 19.8% vs 51.5%. Even so, ablating MILAN's text neurons recovers it strongly (19.8% → 32.7%, +12.8pp) and clearly beats its own importance baseline.

We changed no code between the two architectures, only the trained model. So the editing recipe is model-agnostic across CNNs: wherever the shortcut happens to land, MILAN's captions still locate the neurons that carry it, and removing them helps.

Text-neuron fraction across architectures



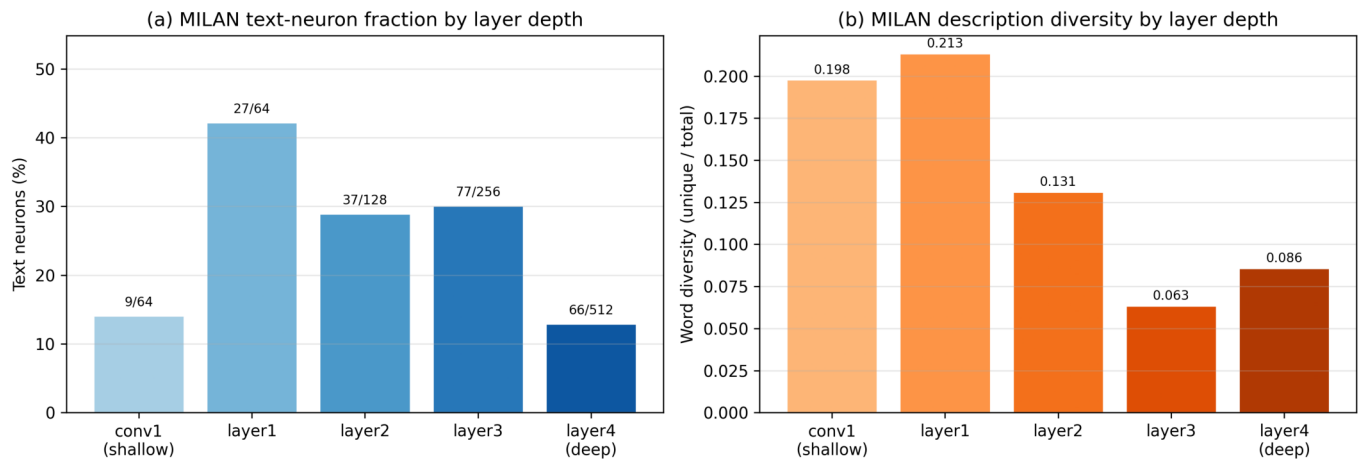
The share of neurons MILAN labels as text-selective, per model. A higher bar means the model leans more heavily on the text shortcut.

ResNet18 (21.1%, 216/1024) and VGG16 (24.3%, 357/1472) are task-trained CNNs and rely on the shortcut heavily. CLIP ViT-B/32 is only 7.9% (61/768) and is used zero-shot — it was never trained on the spurious set.

The contrast previews the robustness result on the CLIP page: how much a model depends on this shortcut is governed less by the raw architecture than by how the model was pretrained.

Where does the shortcut live? (layer depth)

Layer-depth analysis of MILAN descriptions (ResNet18 spurious-text)

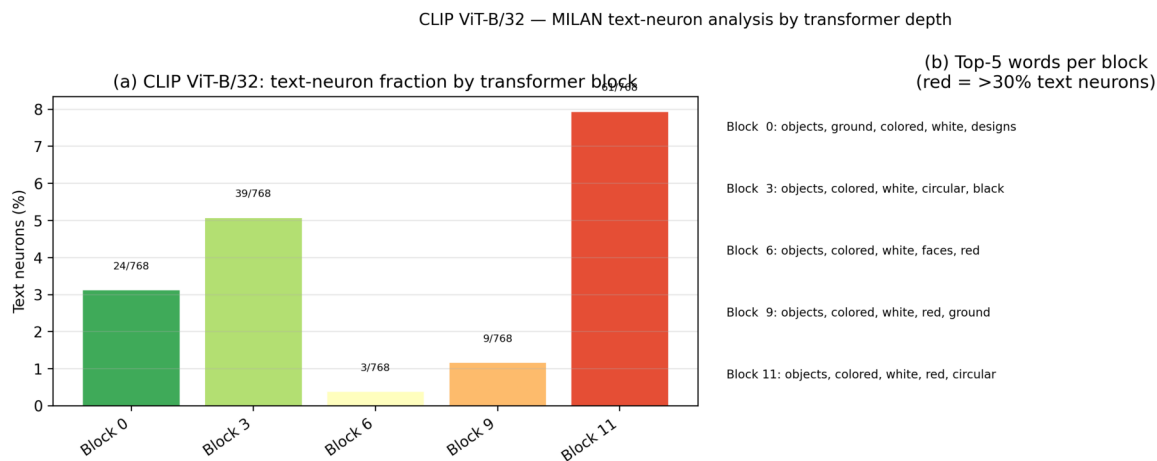


Panel (a): the fraction of text-selective neurons by ResNet18 layer jumps 3.0× from conv1 (14%) to layer1 (42%) — the shortcut appears right after the first residual block — then thins out with depth (layer2–4: 29%, 30%, 13%).

Panel (b): caption diversity falls monotonically with depth, meaning deeper layers produce more repetitive, object-like descriptions.

Together these say the text shortcut crystallizes very early in the network, and that depth mostly adds object structure on top rather than more text-reading features. That is also practically useful: the neurons worth editing out are concentrated in the early-to-mid layers.

Is CLIP more robust? (zero-shot ViT-B/32)



MILAN run on CLIP's vision-transformer blocks, with CLIP used zero-shot (no training on the spurious set). Panel (a) shows the text-neuron fraction by block, peaking at just 7.9% — about $2.7\times$ lower than ResNet18 (21.1%).

Panel (b) lists the top words per block: "objects", "colored", "white" — not "words"/"letters". CLIP simply does not devote many units to reading the overlay.

So susceptibility to this shortcut is structural rather than incidental: CLIP's image-language pretraining yields representations far less reliant on the text overlay than a task-trained CNN — a concrete, measurable robustness advantage, and a fitting close to the generalization story.